

Informe de Proyecto

Simulación basada en Eventos Discretos

Generales

Nombre y apellidos	Melissa Maureen Sales Brito
Grupo	C-311
Asignatura	Simulación
Problema asignado	Problema 5 — Aeropuerto de Barajas
Lenguaje de implementación	Python 3
Repositorio	https://github.com/MaureenSales/Barajas-Airport-Sim.git

Orden del Problema Asignado

En el Aeropuerto de Barajas, se desea conocer cuánto tiempo se encuentran vacías las pistas de aterrizaje. Se conoce que el aeropuerto cuenta con un máximo de 5 pistas de aterrizaje dedicadas a aviones de carga y que se considera que una pista está ocupada cuando hay un avión aterrizando, despegando o cuando se encuentra cargando o descargando mercancía o el abordaje o aterrizaje de cada pasajero.

Se conoce que el tiempo cada avión que arriba al aeropuerto distribuye, mediante una función de distribución exponencial con $\lambda = 20$ minutos.

Si un avión arriba al aeropuerto y no existen pistas vacías, se mantiene esperando hasta que se vacíe una de ellas (en caso de que existan varios aviones en esta situación, pues se establece una suerte de cola para su aterrizaje).

Se conoce además que el tiempo de carga y descarga de un avión distribuye mediante una función de distribución exponencial con $\lambda = 30$ minutos. Se considera además que el tiempo de aterrizaje y despegue de un avión distribuye normal ($N(10,5)$) y la probabilidad de que un avión cargue y/o descargue en cada viaje corresponde a una distribución uniforme.

Además de esto se conoce que los aviones tiene una probabilidad de tener una rotura de 0.1. Así, cuando un avión posee alguna rotura debe ser reparado en un tiempo que distribuye exponencial con $\lambda = 15$ minutos. Las roturas se identifican justo antes del despegue de cada avión.

Igualmente cada avión, durante el tiempo que está en la pista debe recargar combustible y se conoce que el tiempo de recarga de combustible distribuye exponencial $\lambda = 30$ minutos y se comienza justamente cuando el avión aterriza.

Se asume además que los aviones pueden aterrizar en cada pista sin ninguna preferencia o requerimiento.

Simule el comportamiento del aeropuerto por una semana para estimar el tiempo total en que se encuentran vacía cada una de las pistas del aeropuerto.

Principales Ideas Seguidas para la Solución del Problema

1. Identificación del sistema como un modelo de servidores en paralelo extendido

El sistema se reconoció como una variante del modelo de **c servidores en paralelo** estudiado en clase, con $c = 5$ pistas. Sin embargo, a diferencia del modelo clásico donde el servicio es una única fase, aquí el "servicio" de cada avión consta de **múltiples fases con distinta naturaleza temporal**:

- Fases en **paralelo** (ocurren simultáneamente desde que el avión ocupa la pista): aterrizaje, recarga de combustible y carga/descarga de mercancía.
- Fases en **secuencia** (ocurren después): verificación de avería y despegue.

La idea central fue representar cada fase como un **evento independiente** en la lista de eventos futuros, y usar una **barrera de sincronización** que verifique, cada vez que termina una fase paralela, si ya es posible proceder al despegue.

2. Representación del estado con banderas por avión

Para cada avión en pista se mantiene un registro de banderas booleanas (`landing_done`, `refuel_done`, `unload_done`, `repair_done`) que indican qué fases han concluido. El despegue solo se programa cuando las tres fases paralelas están completas. Esto permite que los eventos de las tres fases sean completamente independientes entre sí en la lista de eventos.

3. Cola FIFO global de entrada

Se utiliza una única cola de espera para los aviones que no encuentran pista disponible al llegar, en analogía con el modelo de c servidores en paralelo de los apuntes. Al liberarse una pista, se toma el primer avión en espera (disciplina FIFO) y se inicia su ciclo de servicio.

4. Métrica de tiempo idle acumulada incrementalmente

El tiempo libre de cada pista se acumula de forma incremental: cada vez que una pista queda libre se registra el instante, y cada vez que vuelve a ocuparse se suma la diferencia. Al finalizar la simulación se añade el tiempo libre restante de las pistas que sigan desocupadas. Este mecanismo evita recorrer el historial completo al final.

5. Generación de variables aleatorias desde cero

Todas las muestras aleatorias se generan sin bibliotecas externas, implementando:

- Un **generador congruencial lineal (LCG)** como fuente de uniformes $U(0,1)$.
- El **método de la transformada inversa** para la distribución exponencial.

- El **método de Box-Muller** para la distribución normal.
- **Comparación con umbral** para variables de Bernoulli.

6. Réplicas independientes para estimación estadística

Una única réplica no es suficiente para caracterizar el sistema estocástico. Se ejecutaron **30 réplicas independientes** con semillas distintas para el generador, obteniendo estimaciones promedio por pista con variabilidad controlada.

Modelo de Simulación de Eventos Discretos

Descripción del sistema

Un avión llega al aeropuerto y espera en cola (FIFO) hasta que exista una pista vacía. Según el enunciado, una pista se considera **ocupada** mientras haya un avión aterrizando, despegando, cargando o descargando mercancía, o recargando combustible. Es decir, la pista permanece ocupada durante **todo el tiempo que el avión está en ella**, desde que toca pista hasta que termina el despegue.

Al asignársele una pista, el avión atraviesa las siguientes fases:

1. **Aterrizaje** — Normal($\mu=10$, $\sigma^2=5$) minutos — ocupa la pista desde este momento.
2. **Recarga de combustible** — Exp($\lambda=1/30$) minutos — transcurre en paralelo con el aterrizaje, comenzando desde que el avión toca pista.
3. **Carga/descarga de mercancía** — la probabilidad de que un avión cargue o descargue sigue una distribución Uniforme(0,1): se genera $p \sim U(0,1)$ y el avión carga con esa probabilidad p ; duración Exp($\lambda=1/30$) minutos — transcurre en paralelo con las fases 1 y 2.
4. **Verificación de avería** — probabilidad 0.1 de rotura, detectada justo antes del despegue; reparación Exp($\lambda=1/15$) minutos — se verifica solo cuando las fases 1, 2 y 3 han concluido.
5. **Despegue** — Normal($\mu=10$, $\sigma^2=5$) minutos — comienza tras resolver la avería (si la hubo).

La pista queda libre únicamente al finalizar el despegue. Durante todas las fases anteriores la pista permanece ocupada, lo que es consistente con la definición del enunciado. El horizonte de simulación es $T = 7 \times 24 \times 60 = 10\,080$ minutos (una semana). Al alcanzar T , no se admiten nuevos aviones pero los que ya están en pista o en cola completan su ciclo.

Los tiempos entre llegadas de aviones siguen una distribución Exp($\lambda=1/20$), con una media de 20 minutos entre aviones.

Tipos de eventos

El sistema cuenta con **seis tipos de eventos**:

Evento	Condición de disparo
ARRIBO	Un nuevo avión llega al aeropuerto
FIN_ATERRIAJE	El avión completa la fase de aterrizaje

FIN_COMBUSTIBLE	El avión completa la recarga de combustible
FIN_CARGA	El avión completa la carga/descarga (si aplica)
FIN_REPARACION	El avión completa la reparación (si hay avería)
FIN_DESPEGUE	El avión termina de despegar — pista queda libre

Variables de la simulación

Variables de tiempo:

- **t** — reloj actual de la simulación. Avanza saltando al evento más inminente.
- **tA** — instante del próximo arribo. Si $tA > T$ o no quedan aviones por llegar, $tA = \infty$.
- **LEF** — lista de eventos futuros (min-heap): contiene todos los eventos pendientes de todas las pistas. El próximo evento es siempre $\min(\text{LEF})$.

Variables contadoras:

- **NA** — número total de aviones que han llegado al aeropuerto.
- **ND** — número total de aviones que han completado su ciclo y despegado.

Variables de estado:

- **pistas[i]**, $i = 0..4$ — booleano: True si la pista i está ocupada, False si está libre.
- **cola_espera** — cola FIFO con los identificadores de aviones que esperan pista.
- **estado[id].landing_done** — bandera: aterrizaje completado.
- **estado[id].refuel_done** — bandera: combustible completado.
- **estado[id].unload_done** — bandera: carga/descarga completada (o no requerida).
- **estado[id].repair_done** — bandera: reparación completada (si hubo avería).

Acumuladores de la métrica objetivo:

- **runway_free_time[i]** — tiempo total acumulado que la pista i ha estado libre (minutos).
- **runway_last_freed[i]** — instante en que la pista i quedó libre por última vez.

Inicialización

```

t = 0,  NA = 0,  ND = 0
pistas = [False, False, False, False, False]
cola_espera = []
runway_free_time = [0, 0, 0, 0, 0]
runway_last_freed = [0, 0, 0, 0, 0]

Generar  $t_0 \sim \text{Exp}(1/20)$ 
tA =  $t_0$ 
LEF = { Evento(tA, ARRIBO) }
```

Bucle principal

En cada iteración se extrae el evento de menor tiempo de la LEF:

```
Mientras LEF no esté vacía:
    e ← extraer min(LEF)
    Si e es ARRIBO y e.tiempo > T: descartar, continuar
    t ← e.tiempo
    Ejecutar rutina del evento e
```

Cuando $t_A > T$ se deja de insertar eventos de ARRIBO. El bucle continúa hasta que todas las pistas queden vacías.

Rutinas de cada evento

Evento de arribo — $t_A \leq T$

Condición: el evento más inminente es ARRIBO y $t_A \leq T$.

1. Avanzar el reloj: $t \leftarrow t_A$.
2. Contabilizar: $NA \leftarrow NA + 1$. El valor de NA es el identificador único del avión.
3. Programar el próximo arribo: generar $t_{At} \sim \text{Exp}(1/20)$. Si $t + t_{At} \leq T$: insertar Evento($t + t_{At}$, ARRIBO) en la LEF.
4. Asignar pista:
 - Si existe pista libre i : ejecutar rutina de asignación del avión NA a la pista i .
 - Si todas las pistas están ocupadas: $\text{cola_espera.append}(NA)$.

Rutina de asignación de pista (subrutina)

Cuando se asigna el avión id a la pista k :

1. $\text{pistas}[k] \leftarrow \text{True}$.
2. Acumular tiempo libre previo: $\text{runway_free_time}[k] += t - \text{runway_last_freed}[k]$.
3. Determinar carga/descarga: $\text{unload} \leftarrow \text{Bernoulli}(0.5)$.
 - Si $\text{unload} = \text{False}$: $\text{estado}[id].\text{unload_done} \leftarrow \text{True}$ (no requiere ese evento).
4. Generar y programar en paralelo:
 - $t_{Lt} \leftarrow \max(0, \text{Normal}(10, 5))$; insertar Evento($t + t_{Lt}$, FIN_ATERRIZAJE, id , k).
 - $t_{Rt} \leftarrow \text{Exp}(1/30)$; insertar Evento($t + t_{Rt}$, FIN_COMBUSTIBLE, id , k).
 - Si $\text{unload} = \text{True}$: $t_{Ut} \leftarrow \text{Exp}(1/30)$; insertar Evento($t + t_{Ut}$, FIN_CARGA, id , k).

Evento fin de aterrizaje

1. Avanzar el reloj: $t \leftarrow \text{evento.tiempo}$.
2. $\text{estado}[id].\text{landing_done} \leftarrow \text{True}$.
3. Llamar a la subrutina de intento de despegue.

Evento fin de combustible

1. Avanzar el reloj: $t \leftarrow \text{evento.tiempo}$.
2. `estado[id].refuel_done` $\leftarrow$ True.
3. Llamar a la subrutina de intento de despegue.

Evento fin de carga/descarga

1. Avanzar el reloj: $t \leftarrow \text{evento.tiempo}$.
2. `estado[id].unload_done` $\leftarrow$ True.
3. Llamar a la subrutina de intento de despegue.

Subrutina de intento de despegue

Se ejecuta cada vez que concluye una fase paralela. Solo actúa si las tres condiciones están satisfechas:

```
Si landing_done = True Y refuel_done = True Y unload_done = True:
    averia  $\leftarrow$  Bernoulli(0.1)
    Si averia = True:
        tRep  $\leftarrow$  Exp(1/15)
        Insertar Evento( $t + tRep$ , FIN_REPARACION, id, k) en LEF
    Si averia = False:
        tDt  $\leftarrow$  max(0, Normal(10, 5))
        Insertar Evento( $t + tDt$ , FIN_DESPEGUE, id, k) en LEF
```

Esta subrutina es idempotente: aunque se invoca hasta tres veces (una por fase), solo programa el despegue en la llamada donde se completa la última condición.

Evento fin de reparación

1. Avanzar el reloj: $t \leftarrow \text{evento.tiempo}$.
2. `estado[id].repair_done` $\leftarrow$ True.
3. Generar $tDt \leftarrow \max(0, \text{Normal}(10, 5))$; insertar Evento($t + tDt$, FIN_DESPEGUE, id, k) en LEF.

Evento fin de despegue

Condición: el evento más inminente es FIN_DESPEGUE del avión id en pista k.

1. Avanzar el reloj: $t \leftarrow \text{evento.tiempo}$.
2. $ND \leftarrow ND + 1$.
3. Eliminar registro del avión: borrar `estado[id]`.
4. Liberar la pista: `pistas[k]` $\leftarrow$ False; `runway_last_freed[k]` $\leftarrow$ t.
5. Si `cola_espera` no está vacía:

- $\text{next_id} \leftarrow \text{cola_espera.popleft}()$.
- Ejecutar rutina de asignación de `next_id` a la pista k.

Cierre del sistema

Cuando el evento más inminente es ARRIBO con tiempo > T, se descarta y se establece $tA = \infty$. A partir de ese momento solo se procesan eventos de fases en curso. La simulación concluye cuando todas las pistas están vacías (pistas = [False,...,False]) y la LEF no contiene más eventos de fases activas.

Al finalizar, para cada pista i que esté libre en el instante final t_{fin} :

```
runway_free_time[i] += t_fin - runway_last_freed[i]
```

Consideraciones Obtenidas a partir de la Ejecución de las Simulaciones

Se ejecutaron **30 réplicas independientes** del sistema, cada una simulando una semana completa de operación (T = 10 080 minutos). Los resultados reales obtenidos son los siguientes.

Resultados por pista

Pista	Media idle (min)	% de T	Desv. estándar	Mínimo	Máximo	IC 95% (min)
Pista 1	2 707.7	26.9 %	228.3	2 340.8	3 338.3	[2 622.5 — 2 793.0]
Pista 2	3 731.0	37.0 %	300.4	3 166.7	4 653.6	[3 618.8 — 3 843.2]
Pista 3	5 021.0	49.8 %	406.6	4 256.0	5 729.4	[4 869.2 — 5 172.9]
Pista 4	6 316.4	62.7 %	407.5	5 668.7	6 994.4	[6 164.3 — 6 468.5]
Pista 5	7 620.2	75.6 %	358.5	6 973.2	8 295.7	[7 486.3 — 7 754.1]

Los intervalos de confianza al 95% se calcularon con $t(29, 0.025) = 2.045$.

Resultados globales

Métrica	Valor
Aviones atendidos por semana (media)	504.0
Aviones atendidos — desviación estándar	25.7
Aviones atendidos — mínimo (entre réplicas)	452
Aviones atendidos — máximo (entre réplicas)	547
Aviones en cola al cierre del sistema (media)	0.00
Tiempo idle total (suma 5 pistas, media)	25 396.3 min
Tiempo idle total — desviación estándar	1 424.0 min
Tiempo idle total — mínimo	22 882.7 min
Tiempo idle total — máximo	28 193.9 min

Porcentaje idle promedio por pista	50.4 %
------------------------------------	--------

Consideración 1 — Asimetría entre pistas

Las cinco pistas son físicamente equivalentes, sin embargo presentan tiempos de idle muy distintos: la pista 1 está libre el 26.9% del tiempo mientras que la pista 5 lo está el 75.6%. Esta diferencia de 4 912 minutos (más de 3 días) entre la pista más cargada y la menos cargada es consecuencia directa de la política de asignación: siempre se selecciona la primera pista libre disponible (índice más bajo). La pista 1 es la primera candidata en casi todos los arribos, mientras que la pista 5 solo se ocupa cuando las cuatro anteriores están simultáneamente ocupadas.

Si el objetivo fuera equilibrar el desgaste entre pistas, bastaría con cambiar la política de asignación a **round-robin** o **menor tiempo acumulado de uso**, sin modificar ninguna otra parte del modelo.

Consideración 2 — Estabilidad del sistema

En ninguna de las 30 réplicas quedaron aviones en cola al cierre del sistema (media = 0.00, máximo = 0). Esto confirma que el sistema es **estable**: la tasa de llegadas no supera la capacidad de servicio. Con $\lambda = 1/20$ aviones/minuto y un tiempo de servicio medio aproximado de 77 minutos, la carga teórica es:

$$\rho = \lambda \cdot E[\text{servicio}] / c = (1/20) \cdot 77 / 5 \approx 0.77$$

El sistema opera al 77% de su capacidad en promedio, con margen suficiente para absorber los picos de llegadas sin acumulación permanente de cola.

Consideración 3 — El combustible es el cuello de botella

La recarga de combustible (media 30 min, presente en el 100% de los vuelos) domina sobre el aterrizaje (media 10 min) y sobre la carga/descarga (media 30 min con una probabilidad media de ocurrencia de 0.5, contribuyendo ~15 min en esperanza). En la mayoría de los aviones, la subrutina de intento de despegue se activa en el evento FIN_COMBUSTIBLE, siendo este el último en completarse. Reducir el tiempo de repostaje sería la intervención más efectiva para aumentar el throughput del aeropuerto.

Consideración 4 — Impacto de la avería

Con probabilidad 0.1 y una reparación de media 15 minutos, la avería añade en esperanza 1.5 minutos al ciclo de cada avión. Sobre ~504 aviones por semana, esto suma aproximadamente 756 minutos (12.6 horas) de ocupación adicional en las pistas. El efecto es pequeño en términos relativos (menos del 2% del tiempo de servicio medio) pero no despreciable a escala semanal.

Consideración 5 — Variabilidad entre réplicas

La desviación estándar del tiempo idle total (suma de las 5 pistas) es de 1 424.0 minutos, con un rango de 5 311 minutos entre la réplica con menor idle (22 882.7 min) y la de mayor idle (28 193.9 min). Esto refleja la variabilidad inherente al sistema estocástico: una semana real puede diferir varios miles de minutos del valor esperado. Los intervalos de confianza al 95% muestran estimaciones

precisas (ancho del IC entre 171 y 450 minutos por pista), validando el uso de 30 réplicas.

Consideración 6 — Sensibilidad al parámetro λ

Un incremento en la tasa de llegadas reduciría el tiempo idle de todas las pistas. El umbral de saturación del sistema se alcanza cuando:

$$\lambda \cdot E[\text{servicio}] = c \rightarrow \lambda_{\text{max}} = 5 / 77 \approx 0.065 \text{ aviones/min} \quad (1 \text{ avión cada } \sim 15.4 \text{ min})$$

El valor actual $\lambda = 1/20 = 0.05$ aviones/min está un 23% por debajo de ese umbral. Un aumento del tráfico aéreo de un 20-23% llevaría el sistema al límite de su capacidad, con colas creciendo indefinidamente.